Numeric range algorithms

Document #:	P3732R0 [Latest] [Status]
Date:	2025-06-14
Project:	Programming Language C++
Audience:	SG1,SG9

Reply-to:	Ruslan Arutyunyan <ruslan.arutyunyan@intel.com> Mark Hoemmen <mhoemmen@nvidia.com> Alexey Kukanov <alexey.kukanov@intel.com> Bryce Adelstein Lelbach <brycelelbach@gmail.com> Abhilash Majumder <abmajumder@nvidia.com>

Contents

§ Abstract

We propose `ranges` algorithm overloads (both parallel and non-parallel) for the `<numeric>` header.

§ Authors

- Ruslan Arutyunyan (Intel)
- Mark Hoemmen (NVIDIA)
- Alexey Kukanov (Intel)
- Bryce Adelstein Lelbach (NVIDIA)
- Abhilash Majumder (NVIDIA)

§ 1 What we propose

We propose `ranges` overloads (both parallel and non-parallel) of the following algorithms:

- `reduce`, unary `transform_reduce`, and binary `transform_reduce`;
- `inclusive_scan` and `transform_inclusive_scan`; and
- `exclusive_scan` and `transform_exclusive_scan`.

We also propose adding parallel and non-parallel convenience wrappers:

— `ranges::sum` and `ranges::product` for special cases of `reduce` with addition and multiplication, respectively; and

— `ranges::dot` for the special case of binary `transform_reduce` with transform `multiplies{}` and reduction `plus{}`.

The following sections explain why we propose these algorithms and not others. This relates to other aspects of the design besides algorithm selection, such as whether to include optional projection parameters.

§ 2 Design

§ 2.1 What algorithms to include?

§ 2.1.1 Current set of numeric algorithms

[P3179R8], "C++ Parallel Range Algorithms," is in the last stages of wording review as of the publication date. [P3179R8] explicitly defers adding `ranges` versions of the numeric algorithms. This proposal does that. As such, we focus on the 11 algorithms in [numeric.ops].

— `iota`

— `accumulate`

— `inner_product`

— `partial_sum`

— `adjacent_difference`

— `reduce`

— `inclusive_scan`

— `exclusive_scan`

— `transform_reduce`

— `transform_inclusive_scan`

— `transform_exclusive_scan`

We don't have to add ranges versions of all these algorithms. Several already have a ranges version in C++23, possibly with a different name. Some others could be omitted because they have straightforward replacements using existing views and other `ranges` algorithms. We carefully read the two proposals [P2214R2], "A Plan for C++23 Ranges,"

and [P2760R1], "A Plan for C++26 Ranges," in order to inform our algorithm selections. In some cases that we will explain below, usability and performance concerns led us to disagree with their conclusions.

§ 2.1.2 We propose to include all `*_reduce` and `*_scan` algorithms

§ 2.1.2.1 Summary

We propose

— providing both unary and binary `ranges::transform_reduce` as well as `ranges::reduce`,

— providing `ranges::transform_{in,ex}clusive_scan` as well as `ranges::{in,ex}clusive_scan`, and

— *not* providing projections for any of these algorithms.

§ 2.1.2.2 Do we want `transform_*` algorithms and/or projections?

We start with two questions.

1. Should the existing C++17 algorithms `transform_reduce`, `transform_inclusive_scan`, and `transform_exclusive_scan` have ranges versions, or does it suffice to have ranges versions of `reduce`, `inclusive_scan`, and `exclusive_scan`?
2. Should ranges versions of `reduce`, `inclusive_scan`, and `exclusive_scan` take optional projections, just like `ranges::for_each` and other `ranges` algorithms do?

We use words like "should" because the ranges library doesn't actually *need* `transform_*` algorithms or projections for functional completeness. These questions are about usability and optimization, including the way that certain kinds of ranges constructs can hinder parallelization on different kinds of hardware.

§ 2.1.2.3 Unary transforms, projections, and `transform_view` are functionally equivalent

The above two questions are related, since a projection can have the same effect as a `transform_*` function. This aligns with Section 13.2 of N4128, which explains why `ranges` algorithms take optional projections "everywhere it makes sense."

> Wherever appropriate, algorithms should optionally take *`INVOKE`*-able *projections* that are applied to each element in the input sequence(s). This, in effect, allows users to trivially transform each input sequence for the sake of that single algorithm invocation.

Projecting the input of `reduce` has the same effect as unary `transform_reduce`. Here is an example, in which `get_element` is a customization point object like the one proposed in

[P2769R3], such that `get_element<k>` gets the `k`-th element of an object that participates in the tuple or structured binding protocol.

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
constexpr int init = 3;
auto result_proj =
  std::ranges::reduce(v1, init, std::plus{}, get_element<0>{});
assert(result_proj == 26);
auto result_xform =
  std::ranges::transform_reduce(v1, init, std::plus{}, get_element<0>{});
assert(result_xform == 26);
```

Even without projections, the `transform_*` algorithms can be replaced by a combination of `transform_view` and the non-`transform` algorithm.

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
constexpr int init = 3;
auto result_tv = std::ranges::reduce(
  std::views::transform(v1, get_element<0>{}), init, std::plus{});
assert(result_tv == 26);
```

This applies to scan algorithms as well. [P2214R2] points out that `ranges::transform_inclusive_scan(r, o, f, g)` can be rewritten as `ranges::inclusive_scan(r | views::transform(g), o, f)`. The latter formulation saves users from needing to remember which of `f` and `g` is the transform (unary) operation, and which is the binary operation. Making the ranges version of the algorithm take an optional projection would be exactly equivalent to adding a `transform_*` version that does not take a projection: e.g., `ranges::inclusive_scan(r, o, f, g)` with `g` as the projection would do exactly the same thing as `ranges::transform_inclusive_scan(r, o, f, g)` with `g` as the transform operation.

§ 2.1.2.4 Binary `transform_reduce` is functionally equivalent to `reduce` and `zip_transform_view`

The binary variant of `transform_reduce` is different. Unlike `reduce` and most other numeric algorithms, it takes two input sequences and applies a binary function to the pairs of elements from both sequences. Projections, being unary functions, cannot replace the binary transform function of the algorithm. `transform_view` is similarly of no help unless it is combined with `zip_view` and operates on tuples of elements. `zip_transform_view` is a convenient way to express this combination; applying `reduce` to `zip_transform_view` gives the necessary result (code examples are shown below).

§ 2.1.2.5 Study `ranges::transform` for design hints

Questions about transforms and projections suggest studying `ranges::transform` for design hints. This leads us to two more questions.

1. If transforms and projections are equivalent, then why does `std::ranges::transform` take an optional projection?
2. If binary transform is equivalent to unary transform of a `zip_transform_view`, then why does binary `std::ranges::transform` exist?

2.1.2.5.1 Binary transform

It can help to look at examples. The code below shows the same binary transform computation done in two different ways: without projections and with projections.

`transform` without and with projections

Without projections

```
struct foo {};
std::vector<std::tuple<int, foo,
        std::string>> v1{
  {5, {}, "five"}, {7, {},
        "seven"}, {11, {},
        "eleven"}};
std::vector<std::pair<int,
        std::string>> v2{
  {13, "thirteen"}, {17,
        "seventeen"}, {19,
        "nineteen"}};
std::vector<int>
        out(std::from_range,
        std::views::repeat(0,
        3));;
std::vector<int> expected{65,
        119, 209};

// Without projections: Big,
        opaque lambda
std::ranges::transform(v1, v2,
        out.begin(),
  [] (auto x, auto y) { return
        get<0>(x) * get<0>(y);
        });
assert(out == expected);
```

With projections

```
struct foo {};
std::vector<std::tuple<int, foo,
        std::string>> v1{
  {5, {}, "five"}, {7, {},
        "seven"}, {11, {},
        "eleven"}};
std::vector<std::pair<int,
        std::string>> v2{
  {13, "thirteen"}, {17,
        "seventeen"}, {19,
        "nineteen"}};
std::vector<int>
        out(std::from_range,
        std::views::repeat(0,
        3));;
std::vector<int> expected{65,
        119, 209};

// With projections: More
        readable
std::ranges::transform(v1, v2,
        out.begin(),
  std::multiplies{},
        get_element<0>{},
        get_element<0>{});
assert(out2 == expected);
```

The code without projections using a single big lambda to express the binary operation. Users have to read the big lambda to see what it does. So does the compiler, which can hinder optimization if it's not good at inlining. In contrast, the version with projections lets users read out loud what it does. It also separates the "selection" or "query" part of the transform from the "arithmetic" or "computation" part. The power of the ranges abstraction is that users can factor computation on a range from the logic to iterate over that range. It's natural to extend this separation to selection logic as well.

2.1.2.5.2 Unary transform

It's harder to avoid a lambda, as the function that does an operation, in the unary `transform` case. Most of the named C++ Standard Library arithmetic function objects are binary. Currying them into unary functions in C++ requires either making a lambda (which defeats the purpose somewhat) or using something like `std::bind_front` (which is verbose). On the other hand, using a projection still has the benefit of separating the "selection" part of the transform from the "computation" part.

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
std::vector<int> out(std::from_range, std::views::repeat(0, 3));
std::vector<int> expected{6, 8, 12};

// Unary transform without projection
std::ranges::transform(v1, out.begin(), [] (auto x) { return get<0>(x) +
        1; });
assert(out == expected);

// Unary transform with projection
std::ranges::transform(v1, out.begin(), [] (auto x) { return x + 1; },
        get_element<0>{});
assert(out == expected);

// Unary transform with projection and "curried" plus
std::ranges::transform(v1, out.begin(), std::bind_front(std::plus{}, 1),
        get_element<0>{});
assert(out == expected);
```

§ 2.1.2.6 `reduce`: transforms and projections

We return to the `reduce` examples we showed above, but this time, we focus on their readability.

2.1.2.6.1 Unary `transform_reduce`

A `ranges::reduce` that takes a projection is functionally equivalent to unary `transform_reduce` without a projection. If `ranges` algorithms take projections whenever possible, then the name `transform_reduce` is redundant here. Readers should know that any extra function argument of a ranges algorithm is most likely a projection. Either way – `reduce` with projection, or unary `transform_reduce` – is straightforward to read, and separates selection (`get_element<0>`) from computation (`std::plus`).

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
constexpr int init = 3;

// reduce with projection get_element<0>
auto result_proj =
  std::ranges::reduce(v1, init, std::plus{}, get_element<0>{});
assert(result_proj == 26);
```

```
// transform_reduce with unary transform get_element<0>
auto result_xform =
  std::ranges::transform_reduce(v1, init, std::plus{}, get_element<0>{});
assert(result_xform == 26);

// reduce with transform_view (no projection)
auto result_xv = std::ranges::reduce(
  std::views::transform(v1, get_element<0>{}), init, std::plus{});
assert(result_xv == 26);
```

On the other hand, `ranges` algorithms take projections whenever possible, and `std::ranges::transform` takes a projection. Why can't `transform_reduce` take a projection? For unary `transform_reduce`, this arguably makes the order of operations less clear. The projection happens first, but most users would have to think about that. A lambda or named function would improve readability.

```
struct bar {
  std::string s;
  int i;
};
std::vector<std::tuple<int, std::string, bar>> v{
  { 5,   "five", {"x", 13}},
  { 7,  "seven", {"y", 17}},
  {11, "eleven", {"z", 19}}};
constexpr int init = 3;

// first get bar, then get bar::i
auto result_proj = std::ranges::transform_reduce(
  v, init, std::plus{}, get_element<1>{}, get_element<2>{});
assert(result_proj == 52);

// first get bar, then get bar::i
auto getter = [] (auto t) {
   return get_element<1>{}(get_element<2>{}(t)); // imagine that get_ele-
        ment works for structs
};
auto result_no_proj = std::ranges::transform_reduce(
  v, init, std::plus{}, getter);
assert(result_no_proj == 52);
```

2.1.2.6.2 Binary `transform_reduce`

As we described above, expressing the functionality of binary `transform_reduce` using only `reduce` requires `zip_transform_view` or something like it, making the `reduce`-only version more verbose. Users may also find it troublesome that `zip_view` and `zip_transform_view` are not pipeable: there is no `{v1, v2} | views::zip` syntax, for example. On the other hand, it's a toss-up which version is easier to understand. Users either need to learn what a "zip transform view" does, or they need to learn about `transform_reduce` and know which of the two function arguments does what.

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
```

```
std::vector<std::pair<std::string, int>> v2{
  {"thirteen", 13}, {"seventeen", 17}, {"nineteen", 19}};
constexpr int init = 3;

// reduce with zip_transform_view
auto result_bztv = std::ranges::reduce(
  std::views::zip_transform(std::multiplies{},
    std::views::transform(v1, get_element<0>{}),
    std::views::transform(v2, get_element<1>{})),
  init, std::plus{});
assert(result_bztv == 396);

// binary transform_reduce
auto result_no_proj = std::ranges::transform_reduce(
  std::views::transform(v1, get_element<0>{}),
  std::views::transform(v2, get_element<1>{}),
  init, std::plus{}, std::multiplies{});
assert(result_no_proj == 396);
```

C++17 binary `transform_reduce` does not take projections. Instead, it takes a binary transform function, that combines elements from the two input ranges into a single value. The algorithm then reduces these values using the binary reduce function and the initial value. It's perhaps misleading that this binary function is called a "transform"; it's really a kind of "inner" reduction on corresponding elements of the two input ranges.

One can imagine a ranges analog of C++17 binary `transform_reduce` that takes two projection functions, as in the example below. It's not too hard for a casual reader to tell that the last two arguments of `reduce` apply to each of the input sequences in turn, but that's still more consecutive function arguments than for any other algorithm in the C++ Standard Library. Without projections, users need to resort to `transform_view`, but this more verbose syntax makes it more clear which functions do what.

```
struct foo {};
std::vector<std::tuple<int, foo, std::string>> v1{
  {5, {}, "five"}, {7, {}, "seven"}, {11, {}, "eleven"}};
std::vector<std::pair<std::string, int>> v2{
  {"thirteen", 13}, {"seventeen", 17}, {"nineteen", 19}};
constexpr int init = 3;

// With projections
auto result_proj = std::ranges::transform_reduce(v1, v2, init,
  std::plus{}, std::multiplies{}, get_element<0>{}, get_element<1>{});
assert(result_proj == 396);

// Without projections
auto result_no_proj = std::ranges::transform_reduce(
  std::views::transform(v1, get_element<0>{}),
  std::views::transform(v2, get_element<1>{}),
  init, std::plus{}, std::multiplies{});
assert(result_no_proj == 396);
```

§ 2.1.2.7 Mixed guidance from the current ranges library

The current ranges library offers only mixed guidance for deciding whether `*reduce` algorithms should take projections.

The various `fold_*` algorithms take no projections. Section 4.6 of [P2322R6] explains that the `fold_left_first` algorithm does not take a projection in order to avoid an extra copy of the leftmost value, that would be required in order to support projections with a range whose iterators yield proxy reference types like `tuple<T&>` (as `views::zip` does). [P2322R6] clarifies that `fold_left_first`, `fold_right_last`, and `fold_left_first_with_iter` all have this issue. However, the remaining two `fold_*` algorithms `fold_left` and `fold_right` do not. This is because they never need to materialize an input value; they can just project each element at iterator `iter` via `invoke(proj, *iter)`, and feed that directly into the binary operation. The author of [P2322R6] has elected to omit projections for all five `fold_*` algorithms, so that they have a consistent interface.

A ranges version of `reduce` does not have `fold_left_first`'s design issue. C++17 algorithms in the `reduce` family can copy results as much as they like, so that would be less of a concern here. However, if we ever wanted a `ranges::reduce_first` algorithm, then the consistency argument would arise.

§ 2.1.2.8 `*transform_view` not always trivially copyable even when function object is

Use of `transform_view` and `zip_transform_view` can make it harder for implementations to parallelize `ranges` algorithms. The problem is that both views might not necessarily be trivially copyable, even if their function object is. If a range isn't trivially copyable, then the implementation must do more work beyond just a `memcpy` or equivalent in order to get copies of the range to different parallel execution units.

Here is an example (available on Compiler Explorer as well).

```
#include <ranges>
#include <type_traits>
#include <vector>

// Function object type that acts just like f2 below.
struct F3 {
  int operator() (int x) const {
    return x + y;
  }
  int y = 1;
};

int main() {
  std::vector v{1, 2, 3};

  // operator= is defaulted; lambda type is trivially copyable
  auto f1 = [] (auto x) {
    return x + 1;
  };
  static_assert(std::is_trivially_copyable_v<decltype(f1)>);
```

```
  // Capture means that lambda's operator= is deleted,
  // but lambda type is still trivially copyable
  auto f2 = [y = 1] (auto x) {
    return x + y;
  };
  static_assert(std::is_trivially_copyable_v<decltype(f2)>);

  // decltype(view1) is trivially copyable
  auto view1 = v | std::views::transform(f1);
  static_assert(std::is_trivially_copyable_v<decltype(view1)>);

  // decltype(view2) is NOT trivially copyable, even though f2 is
  auto view2 = v | std::views::transform(f2);
  static_assert(!std::is_trivially_copyable_v<decltype(view2)>);

  // view3 is trivally copyable, though it behaves just like view2.
  F3 f3{};
  auto view3 = v | std::views::transform(f3);
  static_assert(std::is_trivially_copyable_v<decltype(view3)>);

  return 0;
}
```

Both lambdas `f1` and `f2` are trivially copyable, but `std::views::transform(f2)` is *not* trivally copyable. The wording for both `transform_view` and `zip_transform_view` expresses the input function object of type `F` as stored in an exposition-only *`movable-box<F>`* member. `f2` has a capture that gives it a `=deleted` copy assignment operator. Nevertheless, `f2` is still trivially copyable, because each of its default copy and move operations is either trivial or deleted, and its destructor is nontrivial and deleted.

The problem is *`movable-box`*. As [range.move.wrap] 1.3 explains, since `copyable<decltype(f2)>` is not modeled, *`movable-box`*`<decltype(f2)>` provides a nontrivial, not deleted copy assignment operator. This makes *`movable-box`*`<decltype(f2)>`, and therefore `transform_view` and `zip_transform_view`, not trivially copyable.

This feels like a wording bug. `f2` is a struct with one member, an `int`, and a call operator. Why can't I `memcpy` `views::transform(f2)` wherever I need it to go? Even worse, `f3` is a struct just like `f2`, yet `views::transform(f3)` is trivially copyable.

Implementations can work around this in different ways. For example, an implementation of `std::ranges::reduce` could have a specialization for the range being `zip_transform_view<F, V1, V2>` that reaches inside the `zip_transform_view`, pulls out the function object and views, and calls the equivalent of binary `transform_reduce` with them. However, the ranges library generally wasn't designed to make such transformations easy to implement in portable C++. Views generally don't expose their members – an issue that hinders all kinds of optimizations. (For instance, it should be possible for compilers to transform `cartesian_product_view` of bounded `iota_view` into OpenACC or OpenMP multidimensional nested loops for easier optimization, but `cartesian_product_view` does not have a standard way to get at its member view(s).) As a result, an approach based on specializing algorithms for specific view types means that implementations cannot

straightforwardly depend on a third-party ranges implementation for their views. Parallel algorithm implementers generally prefer to minimize coupling of actual parallel algorithms with Standard Library features that don't directly relate to parallel execution.

§ 2.1.2.9 Review

Let's review what we learned from the above discussion.

— In general and particularly for `ranges::transform`, projections improve readability and expose optimization potential, by separating the selection part of an algorithm from the computation part.

— None of the existing `fold_*` `ranges` algorithms (the closest things the Standard Library currently has to `ranges::reduce`) take projections.

— Ranges `reduce` with a projection and unary `transform_reduce` without a projection have the same functionality, without much usability or implementation difference. Ditto for `{in,ex}clusive_scan` with a projection and `transform_{in,ex}clusive_scan` without.

— Expressing binary `transform_reduce` using only `reduce` requires `zip_transform_view` *always*, even if the two input ranges are contiguous ranges of `int`. This hinders readability and potentially also performance.

— A ranges version of binary `transform_reduce` that takes projections is harder to use and read than a version without projections. However, a version without projections would need `transform_view` in order to offer the same functionality. This potentially hinders performance.

§ 2.1.2.10 Conclusions

We propose

— providing both unary and binary `ranges::transform_reduce` as well as `ranges::reduce`,

— providing `ranges::transform_{in,ex}clusive_scan` as well as `ranges::{in,ex}clusive_scan`, and

— *not* providing projections for any of these algorithms.

We conclude this based on a chain of reasoning, starting with binary `transform_reduce`.

1. We want binary `transform_reduce` for usability and performance reasons. (The "transform" of a binary `transform_reduce` is *not* the same thing as a projection.)
2. It's inconsistent to have binary `transform_reduce` without unary `transform_reduce`.
3. Projections tend to hinder usability of both unary and binary `transform_reduce`. If we have unary `transform_reduce`, we don't need `reduce` with a projection.

4. We already have `fold_*` (effectively special cases of `reduce`) without projections, even though some of the `fold_*` algorithms *could* have had projections.

5. If we have other `*reduce` algorithms without projections as well, then the most consistent thing would be for *no* reduction algorithms to have projections.

6. It's more consistent for the various `*scan` algorithms to look and act like their `*reduce` counterparts, so we provide `ranges::transform_{in,ex}clusive_scan` as well as `ranges::{in,ex}clusive_scan`, and do not provide projections for any of them.

§ 2.1.3 We propose convenience wrappers to replace some algorithms

§ 2.1.3.1 `accumulate`

The `accumulate` algorithm performs operations sequentially. Users who want that left-to-right sequential behavior can call C++23's `fold_left`. For users who are not concerned about the order of operations and who want `accumulate`'s default binary operation, we propose parallel and non-parallel convenience wrappers `ranges::sum`.

§ 2.1.3.2 `inner_product`

The `inner_product` algorithm performs operations sequentially. Users who want that left-to-right sequential behavior can call `fold_left`. Note that [P2214R2] argues specifically against adding a ranges analog of `inner_product`, because it is less fundamental than other algorithms.

For users who are not concerned about the order of operations and who want the default binary operations used by `inner_product`, we propose parallel and non-parallel convenience wrappers `ranges::dot`. We call them `dot` and not `inner_product` because inner products are mathematically more general. We specifically mean not just any inner product, but the inner product that is the dot product. Calling them `dot` has the added benefit that they represent the same mathematical computation as `std::linalg::dot`.

§ 2.1.4 Other existing algorithms can be replaced with views

§ 2.1.4.1 `iota`

C++20 has `iota_view`, the view version of `iota`. One can replace the `iota` algorithm with `iota_view` and `ranges::copy`. In fact, one could argue that `iota_view` is the perfect use case for a view: instead of storing the entire range, users can represent it compactly with two integers. There also should be no optimization concerns with parallel algorithms over an `iota_view`. For example, the Standard specifies `iota_view` in a way that does not hinder it from being trivially copyable, as long as its input types are. The iterator type of `iota_view` is a random access iterator for reasonable lower bound types (e.g., integers).

However, `ranges::iota` algorithm was added since C++23, later than`iota_view`. For the sake of completeness we might want to add a parallel variation of it as well. It's only go-

ing to give a syntactic advantage: if users already have `ranges::iota` in their code, parallelizing it would be as simple as adding an execution policy (assuming the iterator/range categories are satisfied).

We do not propose parallel `ranges::iota` in R0. We are seeking for SG9 (Ranges Study Group) feedback.

§ 2.1.4.2 `adjacent_difference`

The `adjacent_difference` algorithm can be replaced with a combination of `adjacent_transform_view` (which was adopted in C++23) and `ranges::copy`. We argue elsewhere in this proposal that views (such as `adjacent_transform_view`) that use a *`movable-box<F>`* member to represent a function object may have performance issues, due to *`movable-box<F>`* being not trivially copyable even for some cases where `F` is trivially copyable. On the other hand, the existing `adjacent_difference` with the default binary operation (subtraction) could be covered with the trivially copyable `std::minus` function object.

In our experience, adjacent differences or their generalization are often used in combination with other ranges. For example, finite-difference methods (such as Runge-Kutta schemes) for solving time-dependent differential equations may need to add together multiple ranges, each of which is an adjacent difference possibly composed with other functions. If users want to express that as a one-pass algorithm, they might need to combine more than two input ranges, possibly using a combination of `transform_views` and `adjacent_transform_views`. This ultimately would be hard to express as a single "`ranges::adjacent_transform`" algorithm invocation. Furthermore, `ranges::adjacent_transform` is necessarily single-dimensional. It could not be used straightforwardly for finite-difference methods for solving partial differential equations, for example. All this makes an `adjacent_transform` algorithm a lower-priority task.

We do not propose `adjacent_transform` for the reasons described above.

§ 2.1.4.3 `partial_sum`

The `partial_sum` algorithm performs operations sequentially. The existing ranges library does not have an equivalent algorithm with this left-to-right sequential behavior, nor do we propose such an algorithm. For users who want this behavior, [P2760R1] suggests a view instead of an algorithm. [P3351R2], "`views::scan`," proposes this view; it is currently in SG9 (Ranges Study Group) review.

Users of `partial_sum` who are not concerned about the order of operations can call `inclusive_scan` instead, which we propose here. We considered adding a convenience wrapper for the same special case of an inclusive prefix plus-scan that `partial_sum` supports. However, names like `partial_sum` or `prefix_sum` would obscure whether this is an inclusive or exclusive scan. Also, we already have `std::partial_sum` that operates in order. Using the same name as a convenient wrapper on top of out-of-order `*_scan`, we pro-

pose in the paper, is misleading. We think it's not a very convenient convenience wrapper if users have to look these aspects up every time they use it.

If WG21 did want a convenience wrapper, one option would be to give this common use case a longer but more explicit name, like `inclusive_sum_scan`.

§ 2.1.5 We don't propose "the lost algorithm" (noncommutative parallel reduce)

The Standard lacks an analog of `reduce` that can assume associativity but not commutativity of binary operations. One author of this proposal refers to this as "the lost algorithm" (in e.g., Episode 25 of "ASDP: The Podcast"). We do not propose this algorithm, but we would welcome a separate proposal to do so.

The current numeric algorithms express a variety of permissions to reorder binary operations.

— `accumulate` and `partial_sum` both precisely specify the order of binary operations as sequential, from left to right. This works even if the binary operation is neither associative nor commutative.

— The various `*_scan` algorithms can reorder binary operations as if they are associative (they may replace `a + (b + c)` with `(a + b) + c`), but not as if they are commutative (they may replace `a + b` with `b + a`).

— `reduce` can reorder binary operations as if they are both associative and commutative.

What's missing here is a parallel analog of `reduce` with the assumptions of `*_scan`, that is, a reduction that can assume associativity but not commutativity of binary operations. Parallel reduction operations with these assumptions exist in other programming models. For example, MPI (the Message Passing Interface for distributed-memory parallel communication) has a function `MPI_Create_op` for defining custom reduction operators from a user's function. `MPI_Create_op` has a parameter that specifies whether MPI may assume that the user's function is commutative.

Users could get that parallel algorithm by calling `*_scan` with an extra output sequence, and using only the last element. However, this requires extra storage.

A concepts-based approach like [P1813R0]'s could permit specializing `reduce` on whether the user asserts that the binary operation is commutative. [P1813R0] does not attempt to do this; it merely specializes `reduce` on whether the associative and commutative operation has a two-sided identity element. Furthermore, [P1813R0] does not offer a way for users to assert that an operation is associative or commutative, because the `magma` (nonassociative) and `semigroup` (associative) concepts do not differ syntactically. One could imagine a refinement of this design that includes a trait for users to specialize on the type of their binary operation, say `is_commutative<BinaryOp>`. This would be analogous to

the `two_sided_identity` trait in [P1813R0] that lets users declare that their set forms a monoid, a refinement of `semigroup` with a two-sided identity element.

This proposal leaves the described algorithm out of scope. We think the right way would be to propose a new algorithm with a distinct name. A reasonable choice of name would be `fold` (just `fold` by itself, not `fold_left` or `fold_right`).

§ 2.1.6 We don't propose `reduce_with_iter`

A hypothetical `reduce_with_iter` algorithm would look like `fold_left_with_iter`, but would permit reordering of binary operations. It would return both an iterator to one past the last input element, and the computed value. The only reason for a reduction to return an iterator would be if the input range is single-pass. However, users who have a single-pass input range really should be using one of the `fold*` algorithms instead of `reduce*`. As a result, we do not propose the analogous `reduce_with_iter` here.

Just like `fold_left`, the `reduce` algorithm should return just the computed value. Section 4.4 of [P2322R6] argues that this makes it easier to use, and improves consistency with other `ranges` algorithms like `ranges::count` and `ranges::any_of`. It is also consistent with [P3179R8]. Furthermore, even if a `reduce_with_iter` algorithm were to exist, `reduce` should not be specified in terms of it. This is for performance reasons, as Section 4.4 of [P2322R6] elaborates for `fold_left` and `fold_left_with_iter`.

§ 2.1.7 We don't propose `reduce_first`

Section 5.1 of [P2760R1] asks whether the Standard Library should have a `reduce_first` algorithm. Analogously to `fold_left_first`, `reduce_first` would use the first element of the range as the initial value of the reduction operation. One application of `reduce_first` is to support binary operations that lack a natural identity element to serve as the initial value. An example would be `min` on a range of `int` values, where callers would have no way to tell if `INT_MAX` represents an actual value in the range, or a fake "identity" element (that callers may get as a result when the range is empty).

We do not propose `reduce_first` here, only outline arguments against and for adding it.

§ 2.1.7.1 Arguments against `reduce_first`

1. [P3179R8] already proposes parallel ranges overloads of `min_element`, `max_element`, and `minmax_element`. Minima and maxima are the main use cases that lack a natural identity element.
2. Users could always implement `reduce_first` themselves, by extracting the first element from the sequence and using it as the initial value in `reduce`.
3. In practice, most custom binary operations have some value that can work like a neutral initial value, even if it's not mathematically the identity.
4. Unlike `fold_left_first*` and `fold_right_last`, the `*reduce` algorithms are unordered. As a result, there is no reason to privilege the first (or last) element of the

range. One could imagine an algorithm `reduce_any` that uses any element of the range as its initial value.

5. For parallel execution, `reduce_first` does not fully address lack of identity, and potentially creates a suboptimal execution flow. See Reduction's initial value vs. its identity element for more detailed analysis.

§ 2.1.7.2 Arguments for `reduce_first`

1. Some equivalent of `reduce_first` can be used as a building block for parallel reduction with unknown identity, if no other solution is proposed.

2. Even though `min_element`, `max_element`, and `minmax_element` exist, users may still want to combine multiple reductions into a single pass, where some of the reductions are min and/or max, while others have a natural identity. As an example, users may want the minimum of an array of integers (with no natural identity), along with the least index of the array element with the minimum value (whose natural identity is zero). This happens often enough that MPI (the Message Passing Interface for distributed-memory parallel computing) has predefined reduction operations for minimum and its index (`MINLOC`) and maximum and its index (`MAXLOC`). On the other hand, even `MINLOC` and `MAXLOC` have reasonable choices of fake "identity" elements that work in practice, e.g., for `MINLOC`, `INT_MAX` for the minimum value and `INT_MAX` for the least array index (where users are responsible for testing that the returned array index is in bounds).

§ 2.2 Range categories and return types

We propose the following.

— Our parallel algorithms take sized random access ranges.

— Our non-parallel algorithms take sized forward ranges.

— Our scans' return type is an alias of `in_out_result`.

— Our reductions just return the reduction value, not `in_value_result` with an input iterator.

[P3179R8] does not aim for perfect consistency with the range categories accepted by existing `ranges` algorithms. The algorithms proposed by [P3179R8] differ in the following ways.

1. [P3179R8] uses a range, not an iterator, as the output parameter (see Section 2.7).

2. [P3179R8] requires that the ranges be sized (see Section 2.8).

3. [P3179R8] requires random access ranges (see Section 2.6).

Of these differences, (1) and (2) could apply generally to all `ranges` algorithms, so we adopt them for this proposal.

Regarding (1), this would make our proposal the first to add non-parallel range-as-output `ranges` algorithms to the Standard. For arguments in favor of non-parallel algorithms taking a range as output, please refer to [P3490R0], "Justification for ranges as the output of parallel range algorithms." (Despite the title, it has things to say about non-parallel algorithms too.) Taking a range as output would prevent use of existing output-only iterators that do not have a separate sized sentinel type, like `std::back_insert_iterator`. However, all the algorithms we propose require at least forward iterators (see below). [P3490R0] shows that it is possible for both iterator-as-output and range-as-output overloads to coexist, so we follow [P3179R8] by not proposing iterator-as-output algorithms here.

Regarding (2), we make the parallel algorithms proposed here take sized random access ranges, as [P3179R8] does. For consistency, we also propose that the output ranges be sized. As a result, any parallel algorithms with an output range need to return both an iterator to one past the last element of the output, and an iterator to one past the last element of the input. This tells callers whether there was enough room in the output, and if not, where to start when processing the rest of the input. This includes all the `*{ex,in}clusive_scan` algorithms we propose.

Difference (3) relates to [P3179R8] only proposing parallel algorithms. It would make sense for us to relax this requirement for the non-parallel algorithms we propose. This leaves us with two possibilities:

— (single-pass) input and output ranges, the most general; or

— (multipass) forward ranges.

The various reduction and scan algorithms we propose can combine the elements of the range in any order. For this reason, we make the non-parallel algorithms take (multipass) forward ranges, even though this is not consistent with the existing non-parallel `<numeric>` algorithms. If users have single-pass iterators, they should just call one of the `fold_*` algorithms, or use the `views::scan` proposed elsewhere. This has the benefit of letting us specify `ranges::reduce` to return just the value. We don't propose a separate algorithm `reduce_with_iter`, as we explain elsewhere in this proposal.

§ 2.3 Constexpr parallel algorithms?

[P2902R1] proposes to add `constexpr` to the parallel algorithms. [P3179R8] does not object to this; see Section 2.10. We continue the approach of [P3179R8] in not opposing [P2902R1]'s approach, but also not depending on it.

§ 2.4 Reduction's initial value vs. its identity element

It's important to distinguish between a reduction's initial value, and its identity element. C++17's `std::reduce` takes an optional initial value `T init` that is included in the terms of the reduction. This is not necessarily the same as the identity element for a reduction,

which is a value that does not change the reduction's result, no matter how many times it is included. The following example illustrates.

```
std::vector<float> v{5.0f, 7.0f, 11.0f};

// Default initial value is float{}, which is 0.0f.
// It is also the identity for std::plus<>, the default operation
float result = std::reduce(v.begin(), v.end());
assert(result == 23.0f);

// Initial value happens to be the identity in this case.
result = std::reduce(v.begin(), v.end(), 0.0f);
assert(result == 23.0f);

// Initial value is NOT the identity in this case.
float result_plus_3 = std::reduce(v.begin(), v.end(), 3.0f);
assert(result_plus_3 == 26.0f);

// Including arbitrarily many copies of the identity element
// does not change the reduction result.
std::vector<float> v2{5.0f, 0.0f, 7.0f, 0.0f, 0.0f, 11.0f, 0.0f};
result = std::reduce(v.begin(), v.end());
assert(result == 23.0f);
result = std::reduce(v.begin(), v.end(), 0.0f);
assert(result == 23.0f);
```

The identity element can serve as an initial value, but not vice versa. This is especially important for parallelism.

§ 2.4.0.1 Initial value matters most for sequential reduction

From the serial execution perspective, it is easy to miss importance of the reduction identity. Let's consider typical code that sums elements of an indexed array.

```
float sum = 0.0f;
for (size_t i = 0; i<array_size; ++i)
    sum += a[i];
```

The identity element `0.0f` is used to initialize the *accumulator* where the array values then sum up. However, if an initial value for the reduction is provided, it replaces the identity in the code above. An implementation of `reduce` does not therefore need to know the identity of its operation when an initial value is provided.

The initial value parameter of `reduce` also lets users express a "running reduction" where the whole range is not available all at once and users need to call `reduce` repeatedly.

§ 2.4.0.2 Identity element matters most for parallel reduction

The situation is different for parallel execution, because there are more than one accumulator to initialize. Any parallel reduction somehow distributes the data over multiple threads of execution, and each one uses a local accumulator for its part of the job. The ini-

tial value can be used to initialize at most one of those; for others, something else is needed.

If an identity `id` for a binary operator `op` is known, then here is a natural way to parallelize `reduce(R, init, op)` over P processors using the serial version as a building block.

1. Partition the range R into P distinct subsequences S_p.
2. On each processor p compute a local result L_p = `reduce(`S_p`, id, op)` (with `id` as the initial value).
3. Reduce over the local results L_p with `init` as the initial value.

It's not the only and not necessarily the best way though. For example, an implementation for the `unseq` policy probably will not call the serial algorithm. Yet it also needs to somehow initialize local accumulators for each SIMD lane.

§ 2.4.0.3 What to do when the identity element is unknown

Then, what happens to a parallel implementation of C++17 `std::reduce` with a user-defined binary operation, where the Standard offers no way to know the operation's identity, if it exists? There are two other ways to initialize local accumulators: either with values from the respective subsequences or with the reduction of two such values.

The type requirements of `std::reduce` seem to assume the second approach, as the type of the result is not required to be copy-constructible.

```
// using random access iterators for simplicity
auto sum = std::move(op(first[0], first[1]));
size_t sz = last - first;
for (size_t i = 2; i < sz; ++i)
    sum = std::move(op(sum, first[i]));
```

While technically doable, this approach is often suboptimal. In many use cases, the iteration space and the data storage are aligned (e.g. to `std::hardware_constructive_interference_size` or to the SIMD width) to allow for more efficient use of HW. The loop bound changes shown above break this alignment, affecting code efficiency.

At a glance, a hypothetical `reduce_first` could be used in an alternative solution where it would be a serial building block in the step (2) above, instead of `reduce` with `id`. But as we noted, such an implementation is not always the best.

§ 2.4.0.4 Other parallel programming models

Other parallel programming models provide all combinations of design options. Some compute only `reduce_first`, some only `reduce`, and some compute both. Some have a way to specify only an identity element, some only an initial value, and some both.

MPI (the Message Passing Interface for distributed-memory parallel communication) has reductions and lets users define custom binary operations. MPI's reductions compute the analog of `reduce_first`. Users have no way to specify either an initial value or an identity for their custom operations.

In the Draft Fortran 2023 Standard, the `REDUCE` clause permits specification of an identity element.

OpenMP lets users specify the identity value (via an *initializer-clause* `initializer(`*initializer-expr*`)`), which is "used as the initializer for private copies of reduction list items" (see the relevant section of the OpenMP 5.0 specification).

Kokkos lets users define the identity value for custom reduction result types, by giving the reducer class an `init(value_type& value)` member function that sets `value` to the identity (see the section on custom reducers in the Kokkos Programming Guide).

oneTBB asks users to specify the identity value as an argument to `parallel_reduce` function template (see the relevant oneTBB specification page).

SYCL lets users specify the identity value by specializing `sycl::known_identity` class template for a custom reduction operation (see the relevant section of the SYCL specification).

The `std::linalg` linear algebra library in the Working Draft for C++26 says, "A value-initialized object of linear algebra value type shall act as the additive identity" ([linalg.reqs.val] 3).

In Python's NumPy library, `numpy.ufunc.reduce` takes optional initial values. If not provided and the binary operation (a "universal function" (ufunc), effectively an elementwise binary operation on a possibly multidimensional array) has an identity, then the initial values default to the identity. If the binary operation has no identity or the initial values are `None`, then this works like `reduce_first`.

§ 2.4.0.5 Conclusions

Based on the above considerations, we conclude that there are good reasons to consider a mechanism for users to explicitly specify the identity element for parallel reduction. There are options of how that could be achieved, of which we list a few.

— Add an optional extra parameter for the value of identity, defaulting to value initialization.

— Change the meaning of the `init` parameter for parallel algorithms to represent identity instead of the initial value.

— Provide a customization point similar to `sycl::known_identity` that also defaults to value initialization but can be specialized for a given operation.

— Similarly to `std::linalg`, require that for numeric parallel algorithms a value-initialized object shall act as the identity element.

At this point, we do not propose any of these options. We would like to hear feedback from SG1 and SG9 on exploring this further.

§ 2.5 `ranges::reduce` design

In this section, we focus on `ranges::reduce`'s design. The discussion here applies generally to the other algorithms we propose.

§ 2.5.1 No default parameters

Section 5.1 of [P2760R1] states:

> One thing is clear: `ranges::reduce` should *not* take a default binary operation *nor* a default initial [value] parameter. The user needs to supply both.

This motivates the following convenience wrappers:

— `ranges::sum(r)` for `ranges::reduce` with `init = range_value_t<R>()` and `plus{}` as the reduce operation;

— `ranges::product(r)` for `ranges::reduce` with `init = range_value_t<R>(1))` and `multiplies{}` as the reduce operation; and

— `ranges::dot(x, y)` for binary `ranges::transform_reduce` with `init = T()` where `T = decltype(declval<range_value_t<X>>() * declval<range_value_t<Y>>())`, `multiplies{}` as the transform operation, and `plus{}` as the reduce operation.

One argument *for* a default initial value in `std::reduce` is that `int` literals like `0` or `1` do not behave in the expected way with a sequence of `float` or `double`. For `ranges::reduce`, however, making its return value type imitate `ranges::fold_left` instead of `std::reduce` fixes that.

§ 2.5.2 For return type, imitate `ranges::fold_left`, not `std::reduce`

Both `std::reduce` and `std::ranges::fold_left` return the reduction result as a single value. However, they deduce the return type differently. For `ranges::reduce`, we deduce the return type like `std::ranges::fold_left` does, instead of always returning the initial value type `T` like `std::reduce`.

[P2322R6], "`ranges::fold`," added the various `fold_*` `ranges` algorithms to C++23. This proposal explains why `std::ranges::fold_left` may return a different reduction type than `std::reduce` for the same input range, initial value, and binary operation. Consider the following example, adapted from Section 3 of [P2322R6] (Compiler Explorer link).

```cpp
#include <algorithm>
#include <cassert>
#include <iostream>
#include <numeric>
#include <ranges>
#include <type_traits>
#include <vector>

int main() {
  std::vector<double> v = {0.25, 0.75};
  {
    auto r = std::reduce(v.begin(), v.end(), 1, std::plus());
    static_assert(std::is_same_v<decltype(r), int>);
    assert(r == 1);
  }
  {
    auto r = std::ranges::fold_left(v, 1, std::plus());
    static_assert(std::is_same_v<decltype(r), double>);
    assert(r == 2.0);
  }
  return 0;
}
```

The `std::reduce` part of the example expresses a common user error. `ranges::fold_*` instead returns "the decayed result of invoking the binary operation with `T` (the initial value) and the reference type of the range." For the above example, this likely expresses what the user meant. It also works for other common cases, like proxy reference types with an unambiguous conversion to a common type with the initial value.

It's notable that `reduce`-like `mdspan` algorithms in [linalg] – `dot`, `vector_sum_of_squares`, `vector_two_norm`, `vector_abs_sum`, `matrix_frob_norm`, `matrix_one_norm`, and `matrix_inf_norm` – all have the same return type behavior as C++17 `std::reduce`. However, the authors of [linalg] expect typical users of their library to prefer complete control of the return type, even if it means they have to type `1.0` instead of `1`. These [linalg] algorithms also have more precise wording about precision of intermediate terms in sums when the element types and the initial value are all floating-point types or specializations of `complex`. (See e.g., [linalg.algs.blas1.dot] 7.) For ranges reduction algorithms, we expect a larger audience of users and thus prefer consistency with `fold_*`'s return type.

§ 2.6 Constraining numeric ranges algorithms

In summary,

- We use the same constraints as `fold_left` and `fold_right` to constrain the binary operator of `reduce` and `*_scan`.
- We imitate C++17 parallel algorithms and [linalg] ([P1673R13]) by using *GENERALIZED_NONCOMMUTATIVE_SUM* and *GENERALIZED_SUM* to describe the behavior of `reduce` and `*_scan`.

— Otherwise, we follow the approach of [P3179R8] ("C++ Parallel Range Algorithms").

[P3179R8], which is in the last stages of wording review, defines parallel versions of many `ranges` algorithms in the C++ Standard Library. (The "parallel version of an algorithm" is an overload of an algorithm whose first parameter is an execution policy.) That proposal restricts itself to adding parallel versions of existing `ranges` algorithms. [P3179R8] explicitly defers adding overloads to the numeric algorithms in [numeric.ops], because these do not yet have ranges versions. Our proposal fills that gap.

WG21 did not have time to propose ranges-based numeric algorithms with the initial set of `ranges` algorithms in C++20. [P1813R0], "A Concept Design for the Numeric Algorithms," points out the challenge of defining ranges versions of the existing parallel numeric algorithms. What makes this task less straightforward is that the specification of the parallel numeric algorithms permits them to reorder binary operations like addition. This matters because many useful number types do not have associative addition. Lack of associativity is not just a floating-point rounding error issue; one example is saturating integer arithmetic. `Ranges` algorithms are constrained by concepts, but it's not clear even if it's a good idea to define concepts that can express permission to reorder terms in a sum.

C++17 takes the approach of saying that parallel numeric algorithms can reorder the binary operations however they like, but does not say whether any reordering would give the same results as any other reordering. The Standard expresses this through the wording "macros" *GENERALIZED_NONCOMMUTATIVE_SUM* and *GENERALIZED_SUM*. (A wording macro is a parameterized abbreviation for a longer sequence of wording in the Standard. We put "macros" in double quotes because they are not necessarily preprocessor macros. They might not even be implementable as such.) Algorithms become ill-formed, no diagnostic required (IFNDR) if the element types do not define the required operations. [P1813R0] instead defines C++ concepts that represent algebraic structures, all of which involve a set with a closed binary operation. Some of the structures require that the operation be associative and/or commutative. [P1813R0] uses those concepts to constrain the algorithms. This means that the algorithms will not be selected for overload resolution if the element types do not define the required operations. It further means that algorithms could (at least in theory) dispatch based on properties like whether the element type's binary operation is commutative. The concepts include both syntactic and semantic constraints.

WG21 has not expressed a consensus on [P1813R0]'s approach. LEWGI reviewed [P1813R0] at the Belfast meeting in November 2019, but did not forward the proposal and wanted to see it again. Two other proposals express something more like WG21's consensus on constraining the numeric algorithms: [P2214R2], "A Plan for C++23 Ranges," [P1673R13], "A free function linear algebra interface based on the BLAS," which defines mdspan-based analogs of the numeric algorithms. Section 5.1.1 of [P2214R2] points out that [P1813R0]'s approach would overconstrain `fold`; [P2214R2] instead suggests just constraining the operation to be binary invocable. This was ultimately the approach taken by the Standard through the exposition-only concepts *`indirectly-binary-left-fold-`*

able and *indirectly-binary-right-foldable*. Section 5.1.2 of [P2214R2] says that `reduce` "calls for the kinds of constraints that [P1813R0] is proposing."

[P1673R13], which was adopted into the Working Draft for C++26 as [linalg], took an entirely different approach for its set of `mdspan`-based numeric algorithms. Section 10.8, "Constraining matrix and vector element types and scalars," explains the argument. Here is a summary.

1. Requirements like associativity are too strict to be useful for practical types. The only number types in the Standard with associative addition are unsigned integers. It's not just a rounding error "epsilon" issue; sums of saturating integers can have infinite error if one assumes associativity.

2. "The algorithm may reorder sums" (which is what we want to say) means something different than "addition on the terms in the sum is associative" (which is not true for many number types of interest). That is, permission for an algorithm to reparenthesize sums is not the same as a concept constraining the terms in the sum.

3. [P1813R0] defines concepts that generalize a mathematical group. These are only useful for describing a single set of numbers, that is, one type. This excludes useful features like mixed precision (e.g., where the result type in `reduce` differs from the range's element type) and types that use expression templates. One could imagine generalizing this to a set of types that have a common type, but this can be too restrictive; Section 5.1.1 of [P2214R2] gives an example involving two types in a fold that do not have a common type.

[P1673R13] says that algorithms have complete freedom to create temporary copies or value-initialized temporary objects, rearrange addends and partial sums arbitrarily, or perform assignments in any order, as long as this would produce the result specified by the algorithm's *Effects* and *Remarks* when operating on elements of a semiring. The `linalg::dot` ([linalg.algs.blas1.dot]) and `linalg::vector_abs_sum` ([linalg.algs.blas1.asum]) algorithms specifically define the returned result(s) in terms of *GENERALIZED_SUM*. Those algorithms do that because they need to constrain the precision of intermediate terms in the sum (so they need to define those terms). In our case, the Standard already uses *GENERALIZED_SUM* and *GENERALIZED_NONCOMMUTATIVE_SUM* to define iterator-based C++17 algorithms like `reduce`, `inclusive_scan`, and `exclusive_scan`. We can just adapt this wording to talk about ranges instead of iterators. This lets us imitate the approach of [P3179R8] in adding ranges overloads.

Our approach combines the syntactic constraints used for the `fold_*` family of algorithms, with the semantic approach of [P1673R13] and the C++17 parallel numeric algorithms. For example, we constrain `reduce`'s binary operation with both *indirectly-binary-left-foldable* and *indirectly-binary-right-foldable*. (This expresses that if the binary operation is called with an argument of the initial value's type `T`, then that argument can be in either the first or second position.) We express what `reduce` does using *GENERALIZED_SUM*.

§ 2.7 Enabling list-initialization for proposed algorithms

Our proposal follows the same principles as described in [P2248R8] paper. We want to enable the use case with constructing `init` from curly braces.

```
#include <cassert>
#include <numeric>
#include <vector>
#include <functional>

int main() {
  std::vector<double> v = {0.25, 0.75};
  auto r = std::ranges::reduce(v, {1}, std::plus());
  assert(r == 2.0);
}
```

Thus, we need to add a default template argument to `T init` in the proposed signatures. While [P2248R8] does not propose a default template parameter for `init` in `<numeric>` header, we want to address this design question from the beginning for the new set of algorithms because `fold_` family already has this feature.

§ 3 Implementation

The oneAPI DPC++ library (oneDPL) has deployment experience. The implementation is done as experimental with the following deviations from this proposal:

— Algorithms do not have constraints

— `reduce` has more overloads (without init and without binary predicate)

— `*_scan` return type is not `in_out_result`

— Convenience wrappers, proposed in the paper are not implemented. The implementation is expected to be trivial, though.

§ 4 Wording

> Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.
>
> Assume that [P3179R8] has been applied to the Working Draft.

§ 4.1 Update feature test macro

> In [version.syn], increase the value of the `__cpp_lib_parallel_algorithm` macro by replacing YYYYMML below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_parallel_algorithm YYYYMML // also in <algorithm>
```

§ 4.2 Change [numeric.ops.overview]

Change [numeric.ops.overview] (the <numeric> header synopsis) as follows.

§ 4.2.1 Add declaration of exposition-only concepts

Add declarations of exposition-only concepts *indirectly-binary-foldable-impl* and *indirectly-binary-foldable* to [numeric.ops.overview] (the <numeric> header synopsis) as follows.

```
template<class F, class T, class I, class U>
  concept indirectly-binary-left-foldable-impl =  // exposition only
    movable<T> && movable<U> &&
    convertible_to<T, U> && invocable<F&, U, iter_reference_t<I>> &&
    assignable_from<U&, invoke_result_t<F&, U, iter_reference_t<I>>>;
```

```
template<class F, class T, class I, class U>
  concept indirectly-binary-foldable-impl =       // exposition only
    movable<T> && movable<U> &&
    convertible_to<T, U> &&
    invocable<F&, U, iter_reference_t<I>> &&
    assignable_from<U&, invoke_result_t<F&, U, iter_reference_t<I>>> &&
    invocable<F&, iter_reference_t<I>, U> &&
    assignable_from<U&, invoke_result_t<F&, iter_reference_t<I>, U>>;
```

```
template<class F, class T, class I>
  concept indirectly-binary-left-foldable =      // exposition only
    copy_constructible<F> && indirectly_readable<I> &&
    invocable<F&, T, iter_reference_t<I>> &&
    convertible_to<invoke_result_t<F&, T, iter_reference_t<I>>,
            decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>> &&
    indirectly-binary-left-foldable-impl<F, T, I,
                    decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>>;

template<class F, class T, class I>
  concept indirectly-binary-right-foldable =    // exposition only
    indirectly-binary-left-foldable<flipped<F>, T, I>;
```

```
template<class F, class T, class I>
  concept indirectly-binary-foldable =           // exposition only
    copy_constructible<F> && indirectly_readable<I> &&
    invocable<F&, T, iter_reference_t<I>> &&
    convertible_to<invoke_result_t<F&, T, iter_reference_t<I>>,
      decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>> &&
    invocable<F&, iter_reference_t<I>, T> &&
    convertible_to<invoke_result_t<F&, iter_reference_t<I>, T>,
      decay_t<invoke_result_t<F&, iter_reference_t<I>, T>>> &&
    indirectly-binary-foldable-impl<F, T, I,
      decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>>;
```

```
template<input_iterator I, sentinel_for<I> S, class T = iter_value_t<I>,
          indirectly-binary-left-foldable<T, I> F>
  constexpr auto fold_left(I first, S last, T init, F f);
```

§ 4.2.2 Add declarations of ranges `reduce` overloads

Add declarations of ranges overloads of `reduce`, `sum`, and `product` algorithms to [numeric.ops.overview] (the <numeric> header synopsis) as follows.

```
template<class ExecutionPolicy, class ForwardIterator, class T, class
        BinaryOperation>
     T  reduce(ExecutionPolicy&&  exec,  //  freestanding-deleted,  see
        [algorithms.parallel.overloads]
                  ForwardIterator first, ForwardIterator last, T init,
        BinaryOperation binary_op);
```

```
  namespace ranges {

  // Non-parallel overloads of reduce

  template<random_access_iterator I,
           sized_sentinel_for<I> S,
           class T = iter_value_t<I>,
           indirectly-binary-foldable<T, I> F>
       constexpr auto reduce(I first, S last, T init, F binary_op) -> /*
        see below */;
  template<sized-random-access-range R,
           class T = range_value_t<I>,
           indirectly-binary-foldable<T, ranges::iterator_t<R>> F>
       constexpr auto reduce(R&& r, T init, F binary_op) -> /* see below
        */;

  // Parallel overloads of reduce

  template<execution-policy Ep,
           random_access_iterator I,
           sized_sentinel_for<I> S,
           class T = iter_value_t<I>,
           indirectly-binary-foldable<T, I> F>
               auto  reduce(Ep&&  exec,  //  freestanding-deleted,  see
        [algorithms.parallel.overloads]
                    I first, S last, T init, F binary_op) -> /* see below
        */;
  template<execution-policy Ep,
           sized-random-access-range R,
           class T = range_value_t<I>,
           indirectly-binary-foldable<T, ranges::iterator_t<R>> F>
               auto  reduce(Ep&&  exec,  //  freestanding-deleted,  see
        [algorithms.parallel.overloads]
                  R&& r, T init, F binary_op) -> /* see below */;

  // Non-parallel overloads of sum

  template<random_access_iterator I,
           sized_sentinel_for<I> S>
    requires /* see below */
      constexpr auto sum(I first, S last) -> /* see below */;
  template<sized-random-access-range R>
    requires /* see below */
      constexpr auto sum(R&& r) -> /* see below */;
```

```
  // Parallel overloads of sum

  template<execution-policy Ep,
           random_access_iterator I,
           sized_sentinel_for<I> S>
    requires /* see below */
                auto   sum(Ep&&   exec,   //   freestanding-deleted,   see
        [algorithms.parallel.overloads]
              I first, S last) -> /* see below */;
  template<execution-policy Ep,
           sized-random-access-range R>
    requires /* see below */
                auto   sum(Ep&&   exec,   //   freestanding-deleted,   see
        [algorithms.parallel.overloads]
              R&& r) -> /* see below */;

  // Non-parallel overloads of product

  template<random_access_iterator I,
           sized_sentinel_for<I> S>
    requires /* see below */
      constexpr auto product(I first, S last) -> /* see below */;
  template<sized-random-access-range R>
    requires /* see below */
      constexpr auto product(R&& r) -> /* see below */;

  // Parallel overloads of product

  template<execution-policy Ep,
           random_access_iterator I,
           sized_sentinel_for<I> S>
    requires /* see below */
              auto  product(Ep&&  exec,  //  freestanding-deleted,  see
        [algorithms.parallel.overloads]
                  I first, S last) -> /* see below */;
  template<execution-policy Ep,
           sized-random-access-range R>
    requires /* see below */
              auto  product(Ep&&  exec,  //  freestanding-deleted,  see
        [algorithms.parallel.overloads]
                  R&& r) -> /* see below */;

  } // namespace ranges
```

```
// [inner.product], inner product
template<class InputIterator1, class InputIterator2, class T>
  constexpr T inner_product(InputIterator1 first1, InputIterator1 last1,
                            InputIterator2 first2, T init);
```

§ 4.2.3 Add declarations of ranges `inclusive_scan`

TODO

§ 4.2.4 Add declarations of ranges `exclusive_scan`

TODO

§ 4.2.5 Add wording for algorithms

TODO

§ 5 References

[P1673R13] Mark Hoemmen, Daisy Hollman,Christian Trott,Daniel Sunderland,Nevin Liber,Alicia KlinvexLi-Ta Lo,Damien Lebrun-Grandie,Graham Lopez,Peter Caday,Sarah Knepper,Piotr Luszczek,Timothy Costa. 2023-12-18. A free function linear algebra interface based on the BLAS.

https://wg21.link/p1673r13

[P1813R0] Christopher Di Bella. 2019-08-02. A Concept Design for the Numeric Algorithms.

https://wg21.link/p1813r0

[P2214R2] Barry Revzin, Conor Hoekstra, Tim Song. 2022-02-18. A Plan for C++23 Ranges.

https://wg21.link/p2214r2

[P2248R8] Giuseppe D'Angelo. 2024-03-20. Enabling list-initialization for algorithms.

https://wg21.link/p2248r8

[P2322R6] Barry Revzin. 2022-04-22. ranges::fold.

https://wg21.link/p2322r6

[P2760R1] Barry Revzin. 2023-12-15. A Plan for C++26 Ranges.

https://wg21.link/p2760r1

[P2769R3] Ruslan Arutyunyan, Alexey Kukanov. 2024-10-16. get_element customization point object.

https://wg21.link/p2769r3

[P2902R1] Oliver Rosten. 2025-04-15. constexpr "Parallel" Algorithms.

https://wg21.link/p2902r1

[P3179R8] Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. 2025-05-18. C++ parallel range algorithms.

https://wg21.link/p3179r8

[P3351R2] Yihe Li. 2025-01-12. views::scan.

https://wg21.link/p3351r2

[P3490R0] Alexey Kukanov, Ruslan Arutyunyan. 2024-11-14. Justification for ranges as the output of parallel range algorithms.

https://wg21.link/p3490r0